Parameter Estimation

Data Science

Parameter Estimation

- least squares
- maximum likelihood method
- maximum a posteriori
- confidence intervals

Statistical Inference

two main goals:

- **parameter estimation**: estimate unknown parameters from data
- **hypothesis testing**: make decisions about assumptions/models (discussed later in context of model evaluation)

What is Parameter Estimation?

assumption:

data generation process

observed data $\{\boldsymbol{x}_i\}_{i=1}^{n}$ sampled from a probability distribution of random variables $\boldsymbol{X}$

$$p_{\boldsymbol{X}}(\boldsymbol{x}|\boldsymbol{\theta})$$

→ model

with unknown parameters $\boldsymbol{\theta}$

goal: estimate $\boldsymbol{\theta}$ from observed data

Notation

mild notation overload
(to be handled by context)

- random variable: X
- vector of several random variables: $\boldsymbol{X}$

- observation of random variable X: x
- observation of random vector $\boldsymbol{X}$: $\boldsymbol{x}$
  - column vector: $\boldsymbol{x}$
  - row vector: $\boldsymbol{x}^{\mathrm{T}}$

features

- multiple observations
  - of X: $x_1, \cdots, x_n$ or $\{x_i\}_{i=1}^n$
  - of $\boldsymbol{X}$: $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_n$ or $\{\boldsymbol{x}_i\}_{i=1}^n$

samples

- data matrix X
  - rows: samples
  - columns: features

- parameter: θ
- vector of parameters $\boldsymbol{\theta}$

- probability that X takes on value x: $p(x)$

probability distribution
abbreviation for $p_X(x)$

Two Types of Parameter Estimation Tasks

unsupervised setup (no labels)

- data: $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_n$
- model: $p(\boldsymbol{x}; \boldsymbol{\theta})$
- goal: fit distribution of data

supervised setup (with labels)

- data: $(\boldsymbol{x}_i, y_i)$ pairs
- model: $p(y|\boldsymbol{x}; \boldsymbol{\theta})$
- goal: predict y from $\boldsymbol{x}$

ML perspective:

unsupervised learning
(e.g., GMM in clustering)

supervised learning

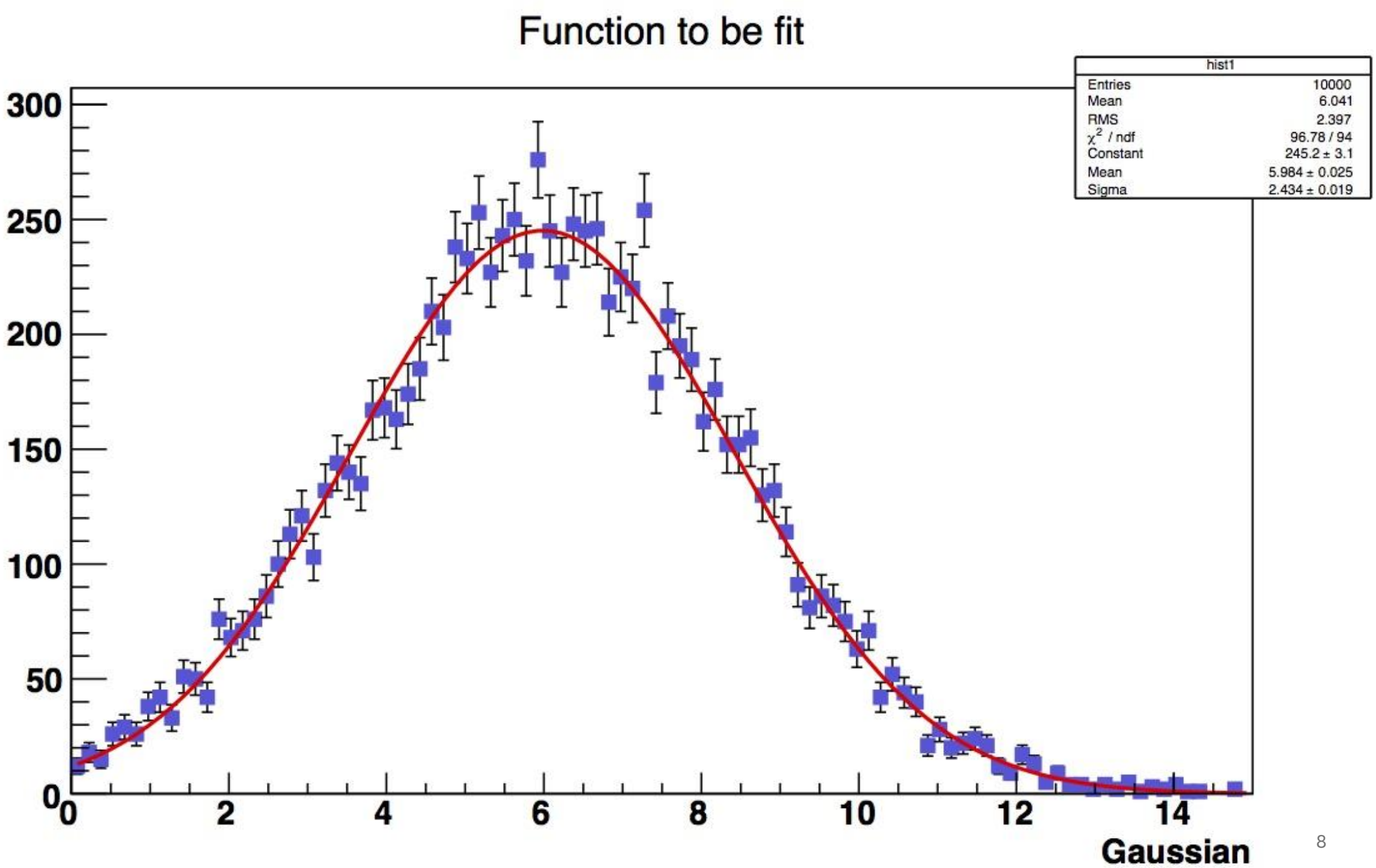
Unsupervised Setup: Examples

mean of a distribution of $X \sim p(x; \boldsymbol{\theta})$

- parameter of interest: $\mu = \mathbb{E}[X]$ (think of μ as θ_1)
- estimated from observed data $x_1, \cdots, x_n$: $\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} x_i$
- examples: mean of Gaussian, probability of success in a Bernoulli

variance of a distribution (e.g., variance of Gaussian)

- parameter of interest: $\sigma^2 = \mathrm{Var}[X]$ (think of σ^2 as θ_2)
- estimated from sample variance

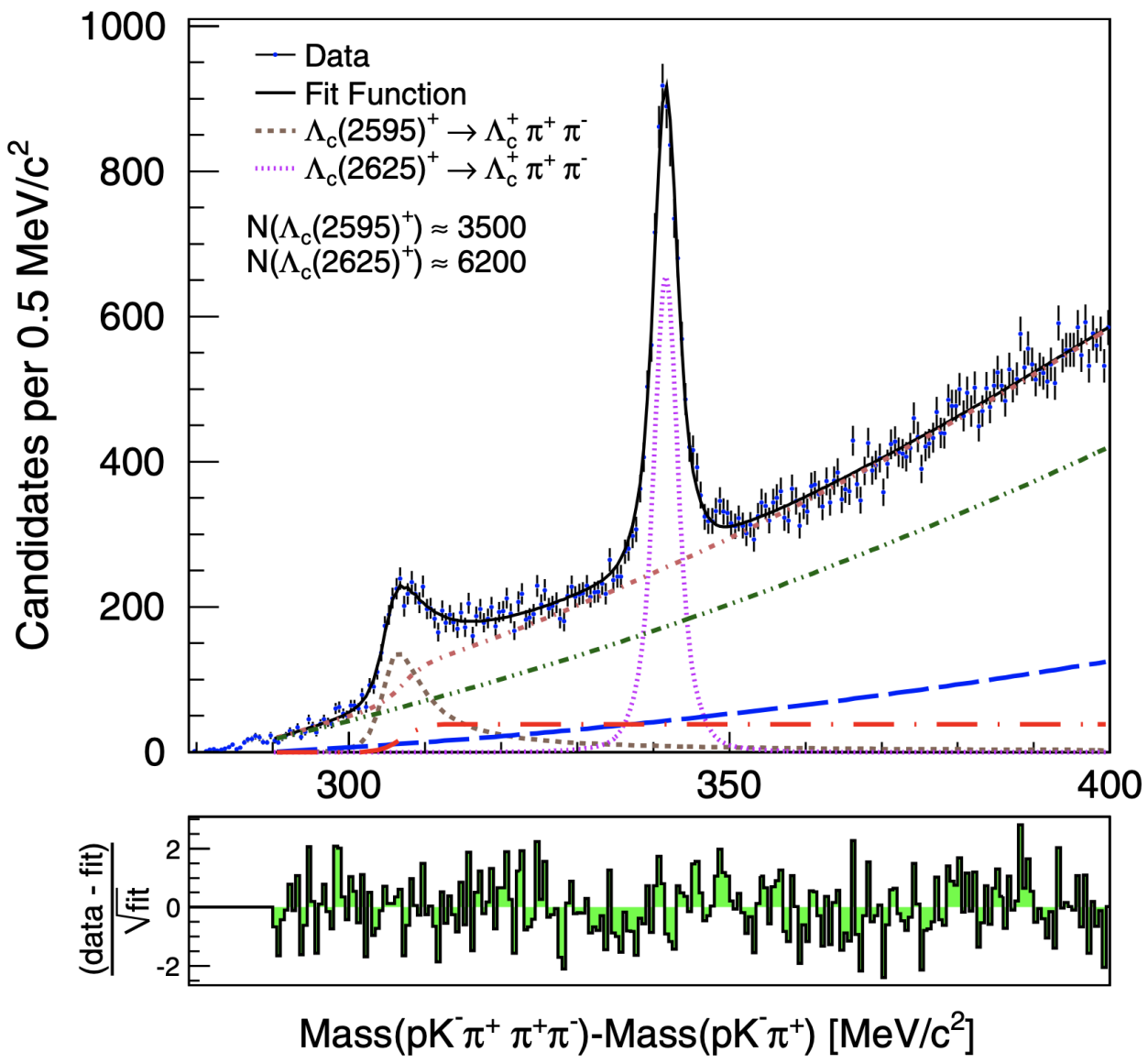

Function to be fit
hist1
Entries 10000
Mean 6.041
RMS 2.397
χ² / ndf 96.78 / 94
Constant 245.2 ± 3.1
Mean 5.984 ± 0.025
Sigma 2.434 ± 0.019
300
250
200
150
100
50
0
0
2
4
6
8
10
12
14
Gaussian

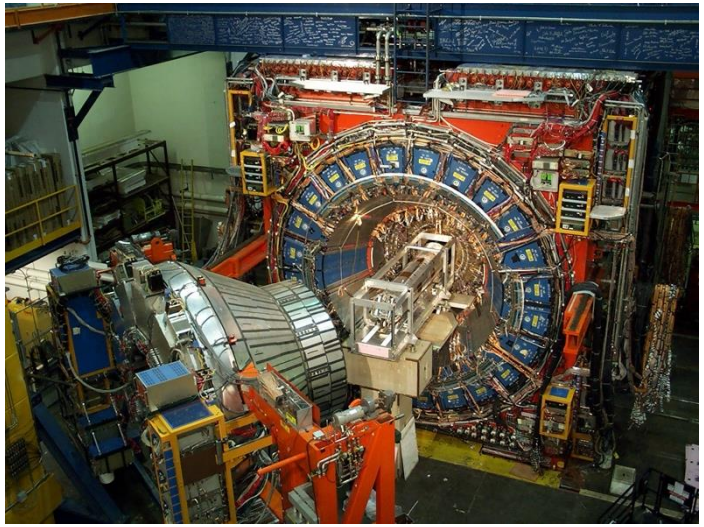
Unsupervised Setup: General Principle

estimate any θ of assumed $p(x; \boldsymbol{\theta})$

find parameter values such that the assumed model best explains the observed data

→ typically requires solving optimization problem (e.g. least squares, maximum likelihood)


Data
Fit Function
$\Lambda_c(2595)^+ \to \Lambda_c^+ \pi^+ \pi^-$
$\Lambda_c(2625)^+ \to \Lambda_c^+ \pi^+ \pi^-$
$N(\Lambda_c(2595)^+) \approx 3500$
$N(\Lambda_c(2625)^+) \approx 6200$
Candidates per 0.5 MeV/c^2
1000
800
600
400
200
0
300
350
400
$\frac{(data - fit)}{\sqrt{fit}}$
2
0
-2
Mass($pK^-\pi^+ \pi^+\pi^-$)-Mass($pK^-\pi^+$) [MeV/c^2]



Supervised Setup

$\boldsymbol{X} \sim p(\boldsymbol{x}), Y|\boldsymbol{X} = \boldsymbol{x} \sim p(y|\boldsymbol{x}; \boldsymbol{\theta})$

→ sampling conditional $(\boldsymbol{x}, y)$ pairs

goal: estimate $\boldsymbol{\theta}$ from observed data $(\boldsymbol{x}_1, y_1), \cdots, (\boldsymbol{x}_n, y_n)$ by fitting $f(\boldsymbol{x}_i; \boldsymbol{\theta})$ to y_i

→ later predict $\hat{y}$ for new $\boldsymbol{x}$ from learned $f\left(\boldsymbol{x}; \widehat{\boldsymbol{\theta}}\right)$

- regression parameters (continuous outputs $y \in \mathbb{R}$)
- classification parameters (discrete outputs $y \in \{1, \cdots, K\}$)

Example: Linear Regression

model (for each observation i):

$$y_i = \underbrace{\boldsymbol{w} \cdot \boldsymbol{x}_i + b}_{f(\boldsymbol{x}_i; \boldsymbol{w}, b)} + \varepsilon_i$$

typically several variables $\boldsymbol{X}$
→ also several parameters $\boldsymbol{w}$

predictive parameters $\boldsymbol{w}, b$
(σ^2 nuisance parameter)

assumption:

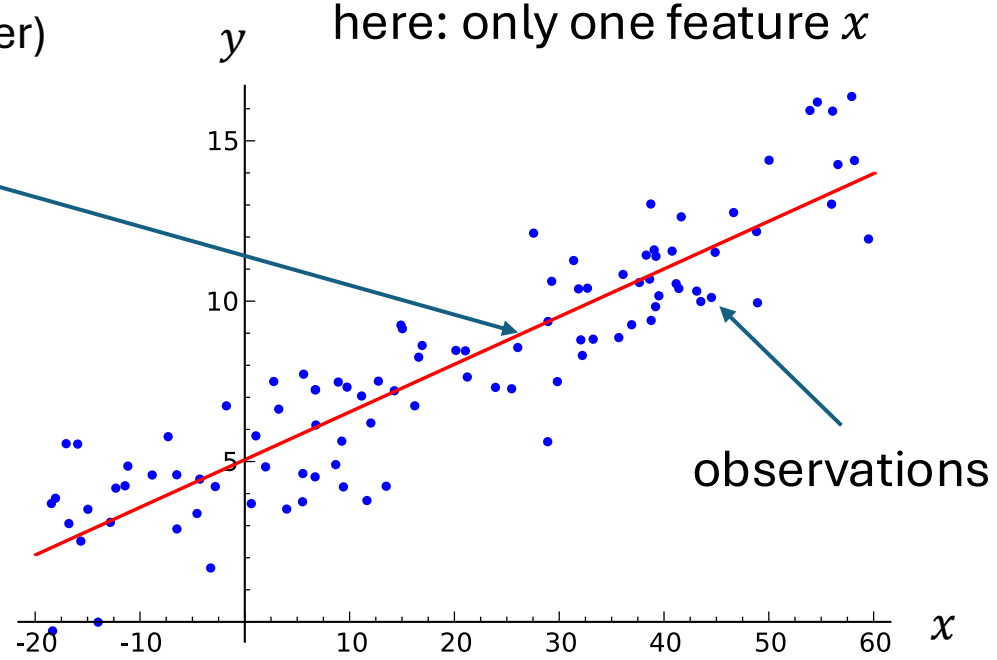
$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

$$\rightarrow p(y|\boldsymbol{x}; w, b, \sigma^2) = \mathcal{N}(\boldsymbol{w} \cdot \boldsymbol{x} + b, \sigma^2)$$

different $p(y|\boldsymbol{x})$ for each $\boldsymbol{x}$,
sharing parameters $\underbrace{w, b, \sigma^2}_{\boldsymbol{\theta}}$

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^{n} \left(y_i - f(\boldsymbol{x}_i; \boldsymbol{w}, b)\right)^2$$

here: only one feature x



observations

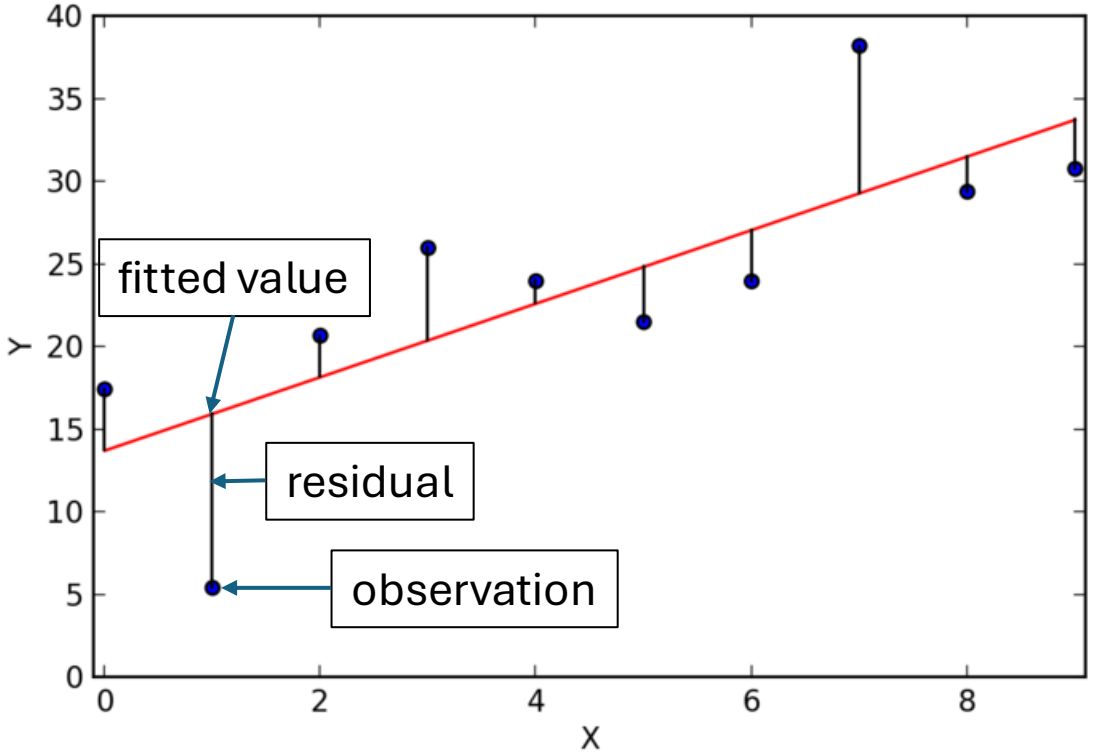
Least Squares Estimation

idea: choose parameters that minimize sum of squared residuals r_i

$$\underset{\boldsymbol{\theta}}{\operatorname{argmin}} \sum_{i=1}^{n} r_i(\boldsymbol{\theta})^2$$

examples for residuals:

- mean estimation: $r_i(\mu) = x_i - \mu$
- regression: $r_i(\boldsymbol{\theta}) = y_i - f(\boldsymbol{x}_i; \boldsymbol{\theta})$



Least Squares Estimation: Minimization

minimization of objective

$$J(\boldsymbol{\theta}) = \sum_{i=1}^{n} r_i(\boldsymbol{\theta})^2$$

candidate solution: need to ensure it is a minimum

first-order optimality condition: set gradient of objective to zero

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \sum_{i=1}^{n} \underbrace{2 r_i(\boldsymbol{\theta}) \nabla_{\boldsymbol{\theta}} r_i(\boldsymbol{\theta})}_{\text{residuals appear multiplied by their sensitivity}} = 0$$

$$\nabla_{\boldsymbol{\theta}} = \begin{bmatrix} \frac{\partial}{\partial \theta_1} \\ \frac{\partial}{\partial \theta_2} \\ \vdots \end{bmatrix}$$

→ $\widehat{\boldsymbol{\theta}}$ satisfies $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = 0$

Linear Least Squares

model (linear in parameters):

$$r_i(\boldsymbol{\theta}) = y_i - \boldsymbol{x}_i^T \boldsymbol{\theta}$$

objective: $J(\boldsymbol{\theta}) = \sum_{i=1}^{n} \left(y_i - \boldsymbol{x}_i^T \boldsymbol{\theta}\right)^2$

matrix form: $J(\boldsymbol{\theta}) = \|\boldsymbol{y} - X\boldsymbol{\theta}\|^2$

new notation: $\boldsymbol{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$ $X = \begin{bmatrix} \boldsymbol{x}_1^T \\ \vdots \\ \boldsymbol{x}_n^T \end{bmatrix}$

linear regression:

$r_i(\boldsymbol{w}, b) = y_i - (\boldsymbol{w} \cdot \boldsymbol{x}_i + b)$

define $\boldsymbol{\theta} = \begin{bmatrix} \boldsymbol{w} \\ b \end{bmatrix}$ and $\tilde{\boldsymbol{x}} = \begin{bmatrix} \boldsymbol{x} \\ 1 \end{bmatrix}$

→ $r_i(\boldsymbol{\theta}) = y_i - \tilde{\boldsymbol{x}}_i^T \boldsymbol{\theta}$

Linear Least Squares: Solution

$$J(\boldsymbol{\theta}) = \|\boldsymbol{y} - X\boldsymbol{\theta}\|^2 = (\boldsymbol{y} - X\boldsymbol{\theta})^T(\boldsymbol{y} - X\boldsymbol{\theta}) = \boldsymbol{y}^T\boldsymbol{y} - \boldsymbol{y}^T X\boldsymbol{\theta} - \boldsymbol{\theta}^T X^T\boldsymbol{y} + \boldsymbol{\theta}^T X^T X\boldsymbol{\theta}$$
$$= \boldsymbol{y}^T\boldsymbol{y} - 2\boldsymbol{\theta}^T X^T\boldsymbol{y} + \boldsymbol{\theta}^T X^T X\boldsymbol{\theta}$$

$$\boldsymbol{y}^T X\boldsymbol{\theta} = (\boldsymbol{\theta}^T X^T\boldsymbol{y})^T = \boldsymbol{\theta}^T X^T\boldsymbol{y}$$

take gradient w.r.t. $\boldsymbol{\theta}$:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = -2X^T\boldsymbol{y} + 2X^T X\boldsymbol{\theta} = 2X^T(X\boldsymbol{\theta} - \boldsymbol{y})$$

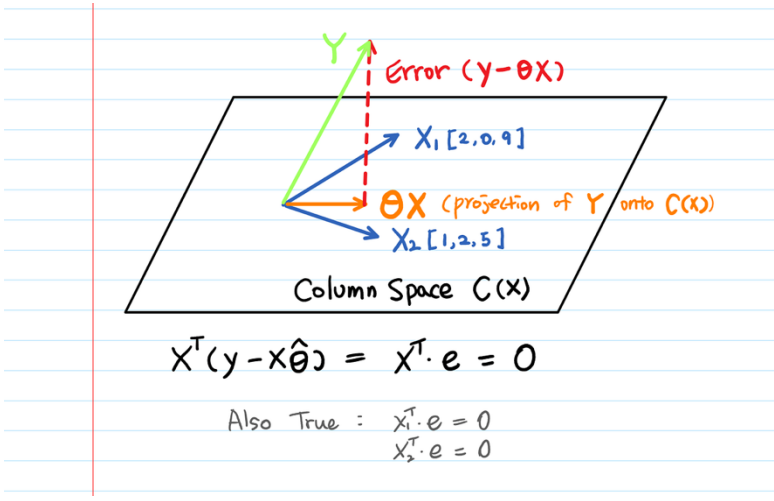
$\nabla_{\boldsymbol{\theta}}(\boldsymbol{\theta}^T A\boldsymbol{\theta}) = (A + A^T)\boldsymbol{\theta}$
here: $A = X^T X$, which is symmetric
→ $\nabla_{\boldsymbol{\theta}}(\boldsymbol{\theta}^T X^T X\boldsymbol{\theta}) = 2X^T X\boldsymbol{\theta}$

set gradient to zero (optimality condition):

$$X^T X\boldsymbol{\theta} = X^T\boldsymbol{y}$$

solution (normal equations):

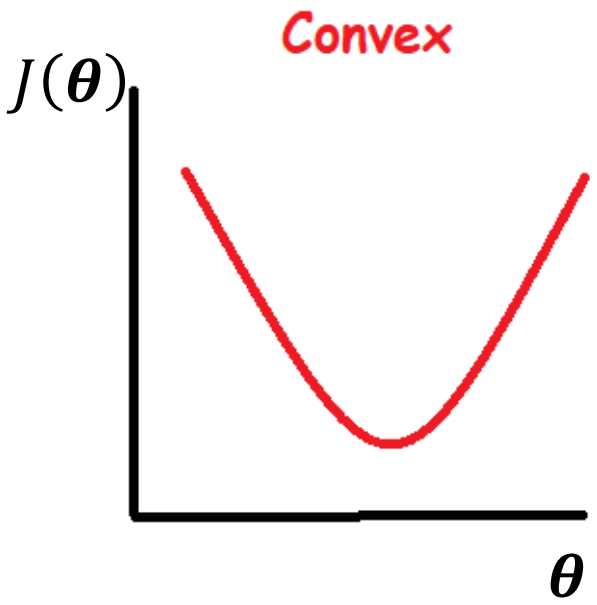
$$\widehat{\boldsymbol{\theta}} = (X^T X)^{-1} X^T\boldsymbol{y}$$

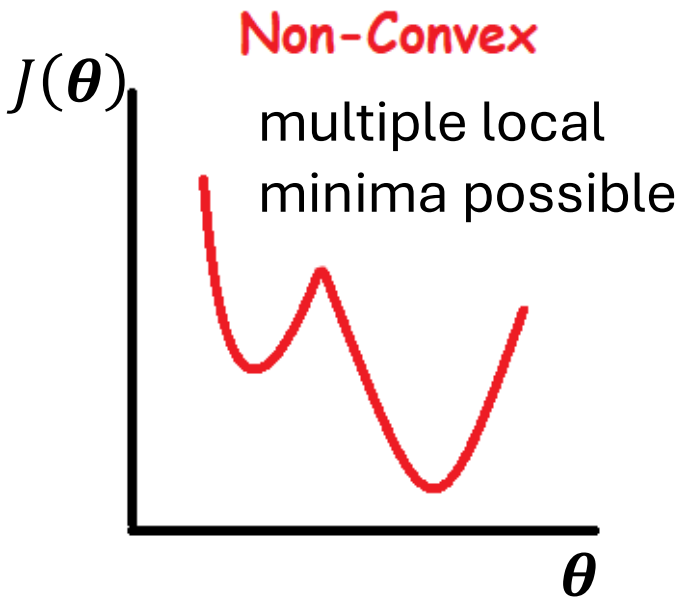


General Nonlinear Case

$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = 0$ gives stationary points

For convex problems (like linear least squares), this is the global minimum.





if $f(\boldsymbol{x}; \boldsymbol{\theta})$ nonlinear in $\boldsymbol{\theta}$: typically no closed-form solution to $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = 0$

→iterative numerical optimization

Gradient Descent

iteratively moving in direction of steepest descent (negative gradient):

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \cdot \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

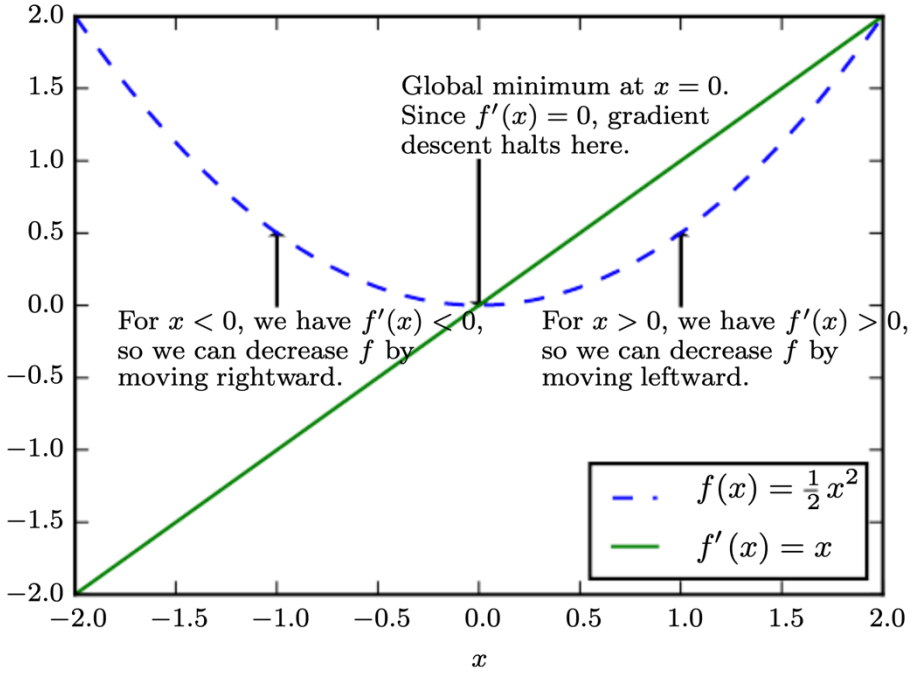
first-order

step size (learning rate): hyperparameter controlling how aggressively to move downhill

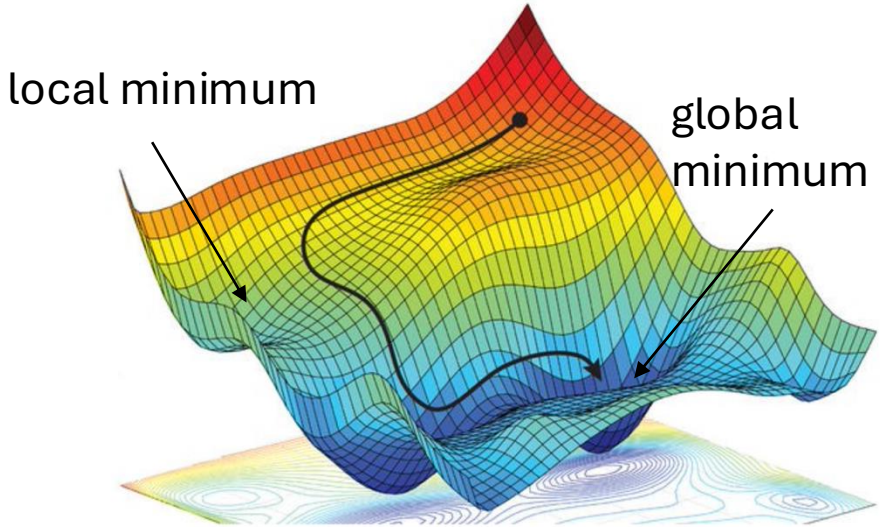
repeatedly take small steps downhill on the loss surface until a minimum is reached

typically use automatic differentiation (autodiff) to calculate gradients (systematic application of chain rule on computation graph)

called backpropagation in neural networks



source



Beyond Gradient Descent

gradient descent is simple and general, but not always the best choice

alternative optimization methods:

- second-order methods → faster convergence, more expensive
  - use curvature (Hessian)
  - Newton's method, quasi-Newton (BFGS)
- stochastic methods (subsampling) → can escape shallow local minima, scales to large datasets, less stable
  - stochastic gradient descent (SGD)
  - mini-batch optimization

} more on this later in deep learning discussion

→ appropriate optimization strategy depends on structure of $J(\boldsymbol{\theta})$

Generalization: Loss Functions

least squares: minimizes sum of squared residuals between predictions and targets (independent of the model used, linear model → linear least squares)

loss functions generalize squared residuals to arbitrary prediction errors (still independent of the model used): $L_i(\boldsymbol{\theta}) = L\big(y_i, f(\boldsymbol{x}_i; \boldsymbol{\theta})\big)$

$L_i(\boldsymbol{\theta})$ quantifies prediction error (e.g., residual) on one sample

further examples:

- absolute error
- cross-entropy (classification)

Cost Function

aggregate (average) losses over data set:

$$J(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} L_i(\boldsymbol{\theta})$$

to be minimized according to model parameters $\boldsymbol{\theta}$

→ objective function

L^2 Regularization (Ridge Regression)

add **penalty term** on parameter L^2 norm to cost function

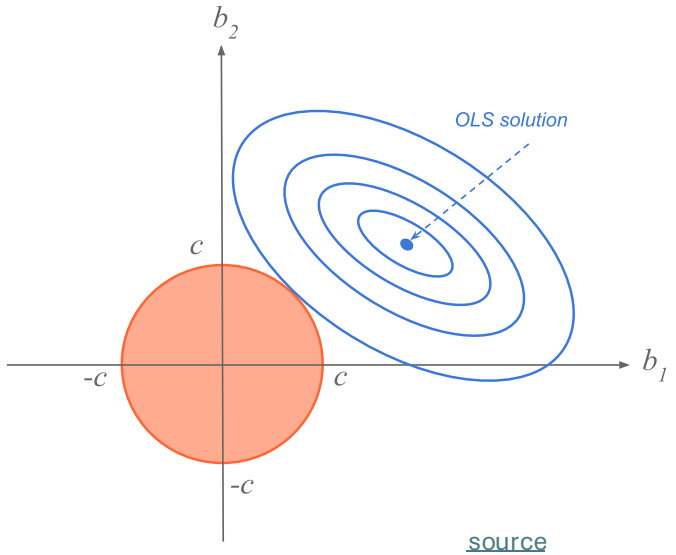
$$\tilde{J}(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \lambda \cdot \boldsymbol{\theta}^{\mathrm{T}}\boldsymbol{\theta}$$

hyperparameter controlling strength of regularization

aka **weight decay** in neural networks

corresponds to imposing constraint on parameters to lie in L^2 region (size controlled by λ)

goal: reduce overfitting from large/unstable parameters



source

Maximum Likelihood Estimation (MLE)

assume data is generated by $p(y|\boldsymbol{x};\boldsymbol{\theta})$

idea: choose $\boldsymbol{\theta}$ to make observed data most likely

likelihood function

$$\widehat{\boldsymbol{\theta}} = \text{argmax}_{\boldsymbol{\theta}} \prod_{i=1}^{n} p(y_i|\boldsymbol{x}_i;\boldsymbol{\theta}) = \text{argmax}_{\boldsymbol{\theta}}\, \mathcal{L}(\boldsymbol{\theta})$$

independent and identically distributed (i.i.d.) samples

→ many loss functions defined from probabilistic assumptions

Side Note: i.i.d. Samples

identically distributed:

each sample drawn from the same distribution: $y_i \sim p(y|\boldsymbol{x}_i; \boldsymbol{\theta})$

→ same model, same parameters for all samples

independent:

→ additive losses: decompose over samples

samples do not influence each other: $p(y_1, \cdots, y_n|X) = \prod_{i=1}^{n} p(y_i|\boldsymbol{x}_i)$

→ knowing one observation gives no information about another

common violations: time series (temporal dependence), spatial data (spatial dependence), text/language (sequential dependence)

Negative Log-Likelihood

often written as log
(means ln, but changing the base
only introduces a constant factor)

log-likelihood function

$$\ell(\boldsymbol{\theta}) = \ln(\mathcal{L}(\boldsymbol{\theta})) = \sum_{i=1}^{n} \ln\big(p(y_i|\boldsymbol{x}_i;\boldsymbol{\theta})\big)$$

- logarithm monotonic function → maximum of ℓ and $\mathcal{L}$ at same $\boldsymbol{\theta}$ values
- most probability distributions (e.g., exponential family) only logarithmically concave (important for optimization)
- sum computationally more convenient than product

negative log-likelihood: minimization instead of maximization

$\widehat{\boldsymbol{\theta}} = \text{argmax}_{\boldsymbol{\theta}}\, \ell(\boldsymbol{\theta}) = \text{argmin}_{\boldsymbol{\theta}}\, \big(-\ell(\boldsymbol{\theta})\big)$ → $\nabla_{\boldsymbol{\theta}}\big(-\ell(\boldsymbol{\theta})\big) = 0$

→ negative log-likelihood: loss function derived from the likelihood

From Gaussian Noise to Least Squares

model assumption: $y_i = f(\boldsymbol{x}_i; \boldsymbol{\theta}) + \varepsilon_i \quad$ with $\quad \varepsilon \sim \mathcal{N}(0, \sigma^2)$
implied distribution: $Y_i | X_i = x_i \sim \mathcal{N}(f(\boldsymbol{x}_i; \boldsymbol{\theta}), \sigma^2)$
likelihood:

$$p(y_i | \boldsymbol{x}_i; \boldsymbol{\theta}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(- \frac{\left(y_i - f(\boldsymbol{x}_i; \boldsymbol{\theta})\right)^2}{2\sigma^2} \right)$$

$$\ell(\boldsymbol{\theta}) = -\frac{1}{2\sigma^2} \sum_{i=1}^{n} \left(y_i - f(\boldsymbol{x}_i; \boldsymbol{\theta})\right)^2 + \text{const}$$

$$\widehat{\boldsymbol{\theta}} = \text{argmax}_{\boldsymbol{\theta}}\, \ell(\boldsymbol{\theta}) = \text{argmin}_{\boldsymbol{\theta}} \sum_{i=1}^{n} \left(y_i - f(\boldsymbol{x}_i; \boldsymbol{\theta})\right)^2$$

→ least squares corresponds to MLE assuming Gaussian noise

different noise assumptions → different loss functions
(other examples: Bernoulli → cross-entropy, Laplace → absolute error)

Two Views on Uncertainty

frequentist perspective

- parameters $\boldsymbol{\theta}$ fixed, but unknown
- data random: uncertainty comes from sampling variability
- estimate $\boldsymbol{\theta}$ using data (likelihood) only (point estimates, potentially with confidence intervals)

Bayesian perspective

- parameters as random variables, following $p(\boldsymbol{\theta}|y, \boldsymbol{x})$
- combine data (likelihood) and prior beliefs to express uncertainty via estimated posterior distributions

Bayes Theorem

H : hypothesis (model)
E : evidence (data)

likelihood function (derived from statistical model)

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)}$$

posterior probability (belief updated with observed evidence)

new data (same for all hypotheses → uninteresting)

prior probability (belief in H before evidence)

Bayes Theorem for Parameter Estimation

data likelihood:
model (function of $\boldsymbol{\theta}$ with data fixed)

$$p(\boldsymbol{\theta}|y,\boldsymbol{x}) \propto p(y|\boldsymbol{x},\boldsymbol{\theta}) \cdot p(\boldsymbol{\theta})$$

posterior:
probability distribution reflecting the effect of data on prior belief about parameters (increasing certainty with new evidence)

prior (e.g., broad Gaussian):
probability distribution reflecting initial belief about uncertain parameters (helps to reduce variance with few data)

to be compared to frequentist point estimates $\widehat{\boldsymbol{\theta}}$

→ Bayesian updating

Maximum A Posteriori (MAP) Estimation

MLE: mode of likelihood as point estimate

$$\widehat{\boldsymbol{\theta}}_{\mathrm{MLE}} = \mathrm{argmax}_{\boldsymbol{\theta}}\, \mathcal{L}(\boldsymbol{\theta}) = \mathrm{argmax}_{\boldsymbol{\theta}} \prod_{i=1}^{n} p(y_i|\boldsymbol{x}_i;\boldsymbol{\theta})$$

uses only the data, ignores prior knowledge

MAP: instead of only likelihood, include prior

$$\widehat{\boldsymbol{\theta}}_{\mathrm{MAP}} = \mathrm{argmax}_{\boldsymbol{\theta}} \prod_{i=1}^{n} p(\boldsymbol{\theta}|y_i,\boldsymbol{x}_i) = \mathrm{argmax}_{\boldsymbol{\theta}} \prod_{i=1}^{n} p(y_i|\boldsymbol{x}_i;\boldsymbol{\theta}) \cdot p(\boldsymbol{\theta})$$

likelihood (MLE)

prior (regularization)

Bayes

log: penalty term

- approximation to Bayesian inference (using mode of posterior distribution)
- MLE special case of MAP estimation with uniform prior

Gaussian Prior → Ridge Regression

$$\widehat{\boldsymbol{\theta}}_{\mathrm{MAP}} = \mathrm{argmin}_{\boldsymbol{\theta}}\left[-\sum_{i=1}^{n}\log\left(p(y_i|\boldsymbol{x}_i;\boldsymbol{\theta})\right) - \log p(\boldsymbol{\theta})\right]$$

log of Gaussian prior $\boldsymbol{\theta}\sim\mathcal{N}(0,\tau^2 I)$:

$$\log p(\boldsymbol{\theta}) = -\frac{1}{2\tau^2}\|\boldsymbol{\theta}^2\| + \mathrm{const}$$

prior belief:
- parameters centered around zero
- large values unlikely

log of Gaussian likelihood:

$$\log\left(p(y_i|\boldsymbol{x}_i;\boldsymbol{\theta})\right) = -\frac{1}{2\sigma^2}\left(y_i - f(\boldsymbol{x}_i;\boldsymbol{\theta})\right)^2 + \mathrm{const}$$

$$\rightarrow \widehat{\boldsymbol{\theta}} = \mathrm{argmin}_{\boldsymbol{\theta}}\left[\sum_{i=1}^{n}\left(y_i - f(\boldsymbol{x}_i;\boldsymbol{\theta})\right)^2 + \underbrace{\frac{\sigma^2}{\tau^2}}_{\lambda}\|\boldsymbol{\theta}^2\|\right]$$

Priors as Regularizers

prior distribution: leveraging information that cannot be found in training data

MAP estimation explains regularization as incorporating prior knowledge into parameter estimation

many regularized estimation strategies can be interpreted as MAP

- L^2 regularization corresponds to MAP with Gaussian prior on parameters
- L^1 regularization corresponds to MAP with isotropic Laplace prior on parameters

MLE: data only

MAP: data + prior (regularization)

From Point Estimates to Uncertainty

two ways to express uncertainty

- Bayesian: credible intervals (probability about $\boldsymbol{\theta}$)
- frequentist: confidence intervals

go back to frequentist view: parameters fixed, uncertainty comes from data

so far point estimates for $\boldsymbol{\theta}$ (single best value)

often need for certainty of these estimates

→ instead of a single value, calculate interval of plausible values

Confidence Intervals (CI)

95% CI: containing the true parameter in 95% of repeated experiments

expressed with α: $100(1-\alpha)\%$

ingredients for CI:

1. estimator $\widehat{\boldsymbol{\theta}}$ (fixed)
2. its variability (standard error)
3. distribution (normal/asymptotic)

general form: $\widehat{\boldsymbol{\theta}} \pm (\text{critical value}) \times (\text{standard error})$

Standard Error (SE)

measures how much the estimate varies due to sampling:
If we repeated the experiment, how much would $\widehat{\boldsymbol{\theta}}$ fluctuate?

example: mean

$$\mathrm{SE}(\mu) = \frac{s}{\sqrt{n}}$$

s: sample standard deviation

n: number of samples

CI depends on data: more data (repetitions of experiment) → smaller SE → more precise estimate (narrower interval)

Critical Value

probabilistic correction factor from Student's t-distribution:

$$t_{n-1,1-\frac{\alpha}{2}}$$

- large n → t approaches standard normal
- small n → heavier tails → wider intervals

SE uses an estimate of the variance, but does not account for the uncertainty of that estimate itself

→ t-distribution accounts for the extra uncertainty

Bootstrapping

idea: treat observed dataset as proxy for the true distribution and simulate new datasets from it

procedure: aproximate the sampling distribution by resampling the data

- given data x $x_1, \cdots, x_n$: draw a bootstrap sample (with replacement)
- compute estimate $\widehat{\boldsymbol{\theta}}_b$
- repeat B times: $\widehat{\boldsymbol{\theta}}_1, \cdots, \widehat{\boldsymbol{\theta}}_B$
- use empirical quantiles for CI: $\left[\text{quantile}_{\alpha/2}, \text{quantile}_{1-\alpha/2}\right]$